

Notebook: linear_regression/linear_regression_on_real_data.ipynb

[--- MARKDOWN CELL ---]

Linear Regression From Scratch on Real Data

In this notebook, we implement Linear Regression from scratch using Gradient Descent and test it on the **California Housing Dataset**.

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
```

[--- CODE CELL ---]

```
# Load the dataset from CSV
df = pd.read_csv('../datasets/california_housing.csv')
df.head()
```

[--- CODE CELL ---]

```
# Prepare features and target
X = df.drop("target", axis=1).values
y = df["target"].values

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[--- MARKDOWN CELL ---]

Implementation From Scratch

[--- CODE CELL ---]

```
class LinearRegressionFromScratch:
    def __init__(self, learning_rate=0.01, epochs=1000):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None
        self.loss_history = []

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        self.loss_history = []

        for i in range(self.epochs):
            # 1. Prediction (Hypothesis)
            y_pred = np.dot(X, self.weights) + self.bias

            # 2. Compute Loss (MSE)
            loss = np.mean((y_pred - y)**2)
            self.loss_history.append(loss)

            # 3. Compute Gradients
            dw = (1/n_samples) * np.dot(X.T, (y_pred - y))
```

```

        db = (1/n_samples) * np.sum(y_pred - y)
        # 4. Update Parameters
        self.weights -= self.lr * dw
        self.bias -= self.lr * db
        if i % 100 == 0:
            print(f"Epoch {i}: Loss {loss:.4f}")
    def predict(self, X):
        return np.dot(X, self.weights) + self.bias

```

[--- CODE CELL ---]

```

# Train the model
model = LinearRegressionFromScratch(learning_rate=0.1, epochs=1000)
model.fit(X_train_scaled, y_train)

```

[--- CODE CELL ---]

```

# Visualize Loss Curve
plt.plot(model.loss_history)
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.title("Training Loss Curve")
plt.show()

```

[--- CODE CELL ---]

```

# Evaluate on Test Data
y_pred = model.predict(X_test_scaled)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Test MSE: {mse:.4f}")
print(f"Test R2 Score: {r2:.4f}")

```

[--- CODE CELL ---]

```

# Compare with scikit-learn's implementation
from sklearn.linear_model import LinearRegression as SkLearnLinearRegression
sk_model = SkLearnLinearRegression()
sk_model.fit(X_train_scaled, y_train)
y_sk_pred = sk_model.predict(X_test_scaled)
print(f"Sklearn MSE: {mean_squared_error(y_test, y_sk_pred):.4f}")
print(f"Sklearn R2 Score: {r2_score(y_test, y_sk_pred):.4f}")

```